NXTGENSEC
SECURING DIGITAL ASSETS

PROGRESS REPORT

Cybersecurity Internship — Session Discussion Notes

- Intern ID : NGS-CS26-KM23
- Project Name : **Keylogger** (Keystroke Monitoring Tool)
- Assigned Mentor Name : **Mouresh**
- Date : 19 July 2026

Topics for Discussion

- **1. Project Progress** — overall status and development phases.
- **2. Code Implementation** — how the tool is built and how it works.
- **3. Features Completed** — implemented capabilities.
- **4. Issues / Blockers** — challenges faced, resolutions, and open items.

Latest code is ready on GitHub (github.com/kartik-reddy-m/key_logger-encoding-decoding) and packaged in Keylogger_KM23.zip; a live demonstration can be given on request.

1. Project Progress

The project — an educational, consent-based keylogger built to demonstrate keystroke capture and, more importantly, how it is detected and defended against — is **functionally complete**. All planned core and advanced features are implemented, tested locally on Windows, documented, and version-controlled on GitHub. Work is now at the demonstration and review stage.

Development phases

Phase	Description	Status
1. Research & Design	Studied the layers where keystrokes can be captured; chose the user-mode API-hook approach (pynput); defined ethical guardrails (consent, local-only, no stealth).	**Completed**
2. Core Implementation	Global keyboard listener, key classification, and local timestamped logging with session markers.	**Completed**
3. Advanced Features	Active-window tagging, log rotation, optional Fernet encryption, and a decryption utility.	**Completed**
4. Documentation	README (setup/usage), technical REPORT (architecture + detection & defense), and the project documentation PDF.	**Completed**
5. Packaging & Version Control	requirements.txt, LICENSE, .gitignore, GitHub repository, and a distributable project zip.	**Completed**
6. Testing & Verification	Verified capture, window markers, rotation, and the encrypt/decrypt round-trip on Windows.	**Completed**

Overall completion: approximately **100%** of the planned scope. Current focus is preparing the live demo and capturing real-run screenshots for the report.

Deliverables ready

- Source code: `keylogger.py`, `decrypt_log.py`
- Support files: `requirements.txt`, `README.md`, `REPORT.md`, `LICENSE`
- GitHub repository + project documentation PDF (with the project zip embedded)

2. Code Implementation

Environment: Python 3.12+ (developed on 3.14). **Dependencies:** pynput >= 1.7.6 (keyboard hook), cryptography >= 42.0.0 (Fernet encryption).

Project structure

File	Responsibility
keylogger.py	Main program: consent gate, keyboard hook, key classification, active-window tagging, and the log writer (rotation + optional encryption).

File	Responsibility
decrypt_log.py	Utility that decrypts an encrypted log using the saved Fernet key.
requirements.txt	Python dependencies.
README.md	Installation and usage guide.
REPORT.md	Architecture, detection, and defense analysis.

How it works

- **Consent & visibility:** `require_consent()` prints a banner and refuses to start until the operator types *I AGREE*; a live keystroke counter shows it is running (no stealth).
- **Keyboard hook:** `pynput.keyboard.Listener` registers a global hook and dispatches events to `on_press` / `on_release` on a background thread.
- **Key classification:** printable keys expose `key.char`; special keys (space, enter, tab, shift…) raise `AttributeError` and are written as readable tokens such as `[shift]`.
- **Active-window context:** `get_active_window_title()` reads the foreground window (Win32 `GetForegroundWindow/GetWindowTextW`, `osascript` on macOS, `xdotool` on Linux); a `--- WINDOW: … ---` marker is logged whenever the app changes.
- **Log writer:** `LogWriter` appends to `logs/keystrokes.log`, rotates it at ~1 MB, and — with `--encrypt` — encrypts each line with a Fernet key in `logs/secret.key`.
- **Stopping:** pressing **ESC** (or Ctrl+C) stops the listener and writes a SESSION END marker with the total keystroke count.
- **Decryption:** `decrypt_log.py` reads each encrypted line and prints the recovered plaintext using the saved key.

Key code excerpt — classifying a keypress

```
def on_press(self, key):
    self._maybe_log_window()
    try:
        char = key.char            # printable key
        if char is not None:
            self.writer.write(char)
    except AttributeError:         # special key (enter, shift, ...)
        special = {keyboard.Key.space: ' ',
                   keyboard.Key.enter: '\n',
                   keyboard.Key.tab: '\t'}
        self.writer.write(special.get(key, f'[{str(key)[4:]}]'))
```

Command-line options

- `--encrypt` — encrypt the log at rest.
- `--no-window` — do not record active-window titles.
- `--i-consent` — skip the interactive prompt (operator accepts terms).

3. Features Completed

Every feature below is implemented and working:

Feature	Detail	Status
Consent gate & visible run	Requires *I AGREE*; prints a live counter.	**Completed**
Local-only logging	Writes to logs/keystrokes.log; no network at all.	**Completed**
Timestamped sessions	SESSION START / END markers with date-time.	**Completed**
Active-window tagging	Records the foreground application per entry.	**Completed**
Special-key handling	Readable tokens like [enter], [tab], [shift].	**Completed**
Log rotation	Rotates at ~1 MB; keeps the last 5 files.	**Completed**
Optional encryption	Per-line Fernet encryption of the log at rest.	**Completed**
Decryption utility	decrypt_log.py restores plaintext with the key.	**Completed**
Cross-platform	Windows, macOS, and Linux support.	**Completed**
Ethical guardrails	No persistence, no stealth, no exfiltration.	**Completed**

4. Issues / Blockers

Challenges faced during development and how each was resolved:

- **Cross-platform active-window detection** — each OS exposes the foreground window differently. *Resolved* with `platform.system()` branching (Win32 via ctypes, osascript, xdotool).
- **Representing non-character keys** — keys like Enter/Shift have no `.char`. *Resolved* by catching `AttributeError` and mapping them to readable tokens.
- **Encryption granularity** — chose *per-line* Fernet tokens instead of whole-file encryption, so logs can be appended and partially decrypted safely.
- **Protecting captured data & keys** — added a `.gitignore` so `logs/` and `secret.key` are never committed, and documented the risk.
- **macOS permissions** — pynput needs *Accessibility / Input Monitoring* access; documented as a setup step (a platform requirement, not a code bug).
- **Environment** — dev machine runs Python 3.14; pinned dependency versions in `requirements.txt` for reproducibility.

Current blockers

None. The project is functionally complete and runs as intended.

Open items / future scope (not blockers)

- Capture real-run screenshots for the report (current images are representative).

- Optional GUI front-end for the demo.
- Optional lab exercise demonstrating exfiltration detection, to strengthen the defensive analysis in REPORT.md.

Ethical note: the main *risk* of a keylogger — misuse — is handled by design through the consent gate, no persistence, no stealth, and no network transmission.

Prepared for the review session — ready to demonstrate the tool live and walk through the code.

Thank You